

Nanobrowser: Architecture, Job Application Lifecycle & LLM Decision Logic

This document provides an in-depth technical explanation of how the **Nanobrowser extension operates**, how it **autonomously applies to LinkedIn Easy Apply jobs**, and exactly **when, where, how, and why it uses Large Language Models (LLMs)**—as well as when LLM invocations are deliberately avoided to eliminate hallucinations and preserve API credits.

Table of Contents

1. High-Level System Architecture
2. End-to-End Job Application Lifecycle
3. Form Question Resolution Pipeline
4. LLM Deep-Dive: When, Where, How, and Why the Model is Invoked
5. When the LLM is Deliberately Skipped (Cost Optimization & Zero Hallucination)
6. Human-In-The-Loop: 5-Minute Unified Batch Questioning
7. Credit Deductions & 100% Automatic Refund Guarantee
8. Executive Summary

1. High-Level System Architecture

Nanobrowser is engineered as a robust, decoupled multi-layer system comprising a **Chrome Extension + Node.js Backend API + AI Bedrock Gateway**:



2. End-to-End Job Application Lifecycle

When you click **"Start Auto Apply"** or **"Apply Current Job"** in the Side Panel, the engine executes the following deterministic lifecycle:

Step 1: Pre-Flight Safety Checks (Before Touching Any Job)

- **Daily Quota Verification:** `DailyQuotaManager.canApplyToday()` ensures daily limits (e.g. 15-20 jobs/day) are respected to protect the user's LinkedIn account from automation bans.
- **Profile Completeness Check:** `validateProfileCompleteness(careerBrain)` verifies that the candidate's resume, full name, email, phone number, and core skills are populated.
- **Credit Balance Verification:** The engine checks backend credit balance (minimum 5 credits required to ensure uninterrupted execution).

Step 2: Dedicated Window & Tab Isolation

To avoid disrupting the candidate's active browsing session, `dedicatedWindowManager` creates an isolated Chrome window. A DevTools detachment listener monitors debugger health; if detached or toggled, execution pauses cleanly and credits are refunded.

Step 3: Job Page Inspection & Login Wall Detection

Upon navigating to the target job URL, `currentPage.extractJobTopCardContext()` evaluates the page:

- **Login Wall Check:** If the candidate is logged out, the run pauses cleanly without spending any credits and prompts the user to log in.
- **Easy Apply Detection:** Verifies the presence of LinkedIn's native "**Easy Apply**" button. External redirect jobs ("Apply on company website") are skipped immediately.

Step 4: Pre-Apply Relevance Fit Check (0 LLM Credits)

The candidate's core skills and background keywords are compared against the job title and requirements using a **purely deterministic algorithmic scoring system** (0 LLM tokens consumed).

Step 5: Modal Opening & Step-by-Step Traversal

`currentPage.clickEasyApplyButton()` launches the application modal. The modal typically contains **1 to 10 progressive steps** (Contact Info → Address → Resume → Screening Questions → Review → Submit). The runner traverses steps using a controlled loop, validating form states before advancing.

3. Form Question Resolution Pipeline

On each modal step, interactive elements are discovered via `page.discoverModalFormFields()`. Fields are then processed through a multi-phase resolution pipeline:

Field Input (e.g. "Notice Period in Days", "AWS Lambda?", "Experience with Python?")



Phase A: FAST & ZERO-COST RESOLUTION (No LLM Calls)



- [Step 1: Exact Rule-Based & Golden Answers Cache]
 - Previously answered questions (e.g. "Notice Period" → "30")
 - Standard profile fields (Phone, Email, LinkedIn Profile URL, City)
 - IF MATCHED: Direct DOM Fill & [MATCHED] Audit Log
- [Step 2: Factual Profile Data Mapping]
 - Current CTC, Expected CTC, Total Years of Experience, Degrees
 - IF MATCHED: Direct DOM Fill & [MATCHED] Audit Log
- [Step 3b: Resume Facts Extraction]
 - Direct factual entity extraction from resume text
- [Step 3c: Numeric Skill-Experience 0-Defaulting]
 - Question: "How many years of work experience with SharePoint / Azure?"
 - Check: Is this skill present in skills list, skillExperience, or resume?
 - IF ABSENT: Auto-defaults to "0" (0 years is the honest minimum)
- [Step 3d: Subjective Experience Yes/No (LLM Inference)]
 - Question: "Have you developed SaaS, CRM, or business applications?"
 - LLM inspects resume excerpt: if clear evidence exists → Answers "Yes"
 - If no evidence exists in resume: Falls through to Phase B (ask_user)
 - NOTE: Unknown tech Yes/No questions are NEVER auto-answered "No" (to prevent false negative claims). The user is prompted instead!



Phase B: BATCHING & OVER-QUALIFICATION SKIP HEURISTIC



- If the step has ≥ 8 Unmatched Screening Fields:
 - ⇒ AUTO-SKIP JOB! ("Over-qualified screening, low candidate fit")
 - ⇒ 0 Credits charged, 100% instant refund, Advances to next job!
- If 1 to 7 Unmatched Fields remain:
 - ⇒ UNIFIED BATCH PROMPT IN SIDE PANEL (Single 5-minute timer)
 - ⇒ Candidate reviews and answers all questions in one scrollable form!
 - ⇒ Submitted answers are saved permanently to Golden Answers.

4. LLM Deep-Dive: When, Where, How, and Why the Model is Invoked

Model & Provider: Amazon Nova-Lite (`amazon.nova-lite-v1:0`) through the backend AWS Bedrock gateway, or the user's custom configured provider (OpenAI GPT-4o-mini, Claude, Gemini Flash).

Core Implementation Files: `dedicatedJobRunner.ts` and `formQuestionResolver.ts` .

Situation 1: Subjective Experience Yes/No Inference (Step 3d)

- **File & Method:** `formQuestionResolver.ts` → `inferSubjectiveExperienceFromResume()`
- **WHEN Invoked:** When an experience-based Yes/No question cannot be answered from structured profile keys, but can be answered from descriptions of past projects in the candidate's resume.
Examples: "Have you developed enterprise SaaS applications?", "Do you have experience designing microservices architectures?"

- **HOW Invoked:** The LLM receives the candidate's **resume excerpt (up to 4,000 characters)**, **skills list**, and the **target question** with a strict evidence-first JSON prompt:

```
{
  "hasEvidence": true,
  "answer": "Yes",
  "evidence": "Built multi-tenant SaaS dashboard at previous company"
}
```

- **WHY Invoked:** To avoid pausing the automation for standard project questions that the resume clearly confirms. If the resume does not substantiate the claim, the model returns `hasEvidence: false` and cleanly defers to the user.

Situation 2: Ambiguous Semantic Field Mapping (Step 3)

- **File & Method:** `formQuestionResolver.ts` → `matchWithLLM()`
- **WHEN Invoked:** When a recruiter phrases a standard factual question using idiosyncratic, non-standard wording that evades rule-based regex patterns.
Examples: "What remuneration are you currently drawing per annum?" (Current CTC), "How soon can you come on board?" (Notice Period).
- **HOW Invoked:** A summarized dictionary of available profile keys is provided to the LLM to identify the underlying intent.
- **WHY Invoked:** To prevent brittle keyword mismatches from failing form submissions while ensuring the correct profile data is inserted.

Situation 3: Free-Text Narrative Generation

- **File & Method:** `formQuestionResolver.ts` → `generateFreeTextWithLLM()`
- **WHEN Invoked:** When a form presents an open-ended textarea without selectable options.
Examples: "Why are you interested in joining our company?", "Describe your most challenging engineering problem."
- **HOW Invoked:** The candidate's `backgroundNarrative`, target job title, and company name are passed to produce a concise (2-3 sentences), professional response.
- **WHY Invoked:** Open-ended narrative questions cannot be statically hardcoded; responses must be tailored to the specific position.

5. When the LLM is Deliberately Skipped (Cost Optimization & Zero Hallucination)

Invoking LLMs indiscriminately causes three critical drawbacks: high API token costs, network latency (1-2s delay per field), and the risk of hallucinating sensitive compliance facts. Nanobrowser enforces a strict **Zero-Invention & Deterministic-First Policy**:

Question Type	Resolution Mechanism	LLM Used?	Credit Cost
Contact Details (Phone, Email, LinkedIn URL, City)	Direct CareerBrain profile lookup	X NO	0
Previously Answered Questions	<code>goldenAnswers</code> memory cache (Instant recall)	X NO	0
Unknown Numeric Skill (e.g. "Years with SharePoint?")	Auto-defaults to <code>0</code> (Step 3c - honest minimum)	X NO	0
Unknown Tech Yes/No (Not in resume)	Batched <code>ask_user</code> / ≥ 8 Skip Heuristic	X NO	0
Legal Compliance (Visa Sponsorship, Background Checks)	Strict profile lookup / Batched <code>ask_user</code>	X NO	0
Notice Period / CTC Format Adaptation	Deterministic regex conversion (<code>adaptAnswerToFieldFormat</code>)	X NO	0
Excessive Screening Forms (≥ 8 Unmatched Fields)	Over-qualification skip heuristic (Low candidate fit)	X NO	0

6. Human-In-The-Loop: 5-Minute Unified Batch Questioning

When a question cannot be resolved from existing profile keys, golden answers, or resume evidence (such as unlisted technical skills, confidential compliance questions, or custom client requirements):

- **Legacy Behavior (Deprecated):** Each unresolved question triggered an isolated popup with an individual 3-minute timer (10 questions = 10 sequential interruptions).
- **Modern Batched Flow:**
 - All fields on the modal step are first evaluated in Phase A (populating known profile data, numeric 0-defaults, and resume matches).
 - Unresolved fields (including unlisted tech Yes/No questions) are aggregated into a **Unified Batch Form in the Side Panel**.
 - A **single combined 5-minute countdown timer** is initiated.
 - The candidate reviews all questions simultaneously in a scrollable card and clicks **"Submit All Answers & Continue Application"**.
 - Submitted answers are injected directly into the DOM and permanently saved to `goldenAnswers` so the candidate is never asked again.

7. Credit Deductions & 100% Automatic Refund Guarantee

When Are Credits Deducted?

Credits are deducted strictly for billable LLM tokens consumed during necessary AI inferences. Nanobrowser charges **zero flat fees** for running the automation loop or filling deterministic fields.

When is a 100% Instant Refund Issued?

If an application cannot be successfully submitted and verified, the engine immediately calls `backendApiClient.refundCredits(jobRunId)` to restore all deducted credits to the user's account:

- Job Skipped (e.g. `≥ 8 unmatched fields` low candidate fit).
- Candidate clicks the Stop Application button.
- Dedicated browser window closed by user.
- LinkedIn prompts for phone verification or CAPTCHA.
- DevTools debugger detached or network connection dropped.
- Form validation errors that cannot be cleared.

8. Executive Summary

- **Deterministic First:** Over 90% of form fields resolve in 0 milliseconds without consuming LLM tokens.
- **Targeted LLM Invocations:** The model is reserved strictly for subjective resume verification and narrative generation.
- **Safe Defaults:** Unknown numeric skill experience defaults to `0` (the honest minimum). Technology Yes/No questions are never guessed to prevent under-representing genuine experience.
- **Streamlined Batch UX:** Unresolved questions appear together in a single 5-minute batch form.
- **Zero Financial Risk:** Incomplete, paused, or skipped applications incur 0 credit costs with automatic 100% refunds.